Dolphin Framework: Absolute Master Guide (100+ Pages Equivalent) 🐬 NP Latest Version: v2.0 | Updated: 2026-04-05 | License: MIT

यो डकुमेन्ट Dolphin Framework को आधिकारिक र विस्तृत गाइड हो। यसले तपाईंलाई एउटा साधारण कोड लेख्ने डेभलपरबाट "Framework Master" बनाउन मद्दत गर्नेछ।

विषय सूची (Table of Contents)

०. परिचय र दर्शन

१. सेटअप र वातावरण

२. कोर सर्भर इन्जिन

३. Unified Context (ctx) Deep Dive

४. एडभान्स्ड राउटिङ

५. मिडलवेयर आर्किटेक्चर

६. डेटाबेस एड्याप्टर

७. Auth मोड्युल मास्टरक्लास

८. अटोमेटेड CRUD इन्जिन

९. Zod भ्यालिडेसन र सुरक्षा

१०. रियल-वर्ल्ड प्रोजेक्ट: DolphinStore API

११. स्केलिङ र पर्फर्मेन्स

१२. टेस्टिङ र डेभप्स

१३. RealtimeCore v2.0: रियलटाइम पब/सब मास्टरक्लास (पूर्ण अपडेटेड)

१३.१ v2.0 का नयाँ फिचरहरू

१३.२ Device Management (डिभाइस व्यवस्थापन)

१३.३ High-Frequency Messaging (pubPush/subPull)

१३.४ File Transfer with Resume (pubFile/subFile)

१३.५ P2P Streaming

१३.६ Private Messaging

१३.७ Basic Pub/Sub (Retro-compatible)

१३.८ पूर्ण उदाहरण: हस्पिटल मोनिटरिङ सिस्टम

१४. इन्डिपेन्डेन्ट राउटिङ

१५. स्वतन्त्र अटो-स्वैगर जेनेरेसन

१६. API रेफरेन्स

०. परिचय र दर्शन (Introduction & Philosophy) Dolphin किन जन्मियो? ब्याकइन्ड डेभलपमेन्टको दुनियाँमा एक्सप्रेस (Express) सबैभन्दा लोकप्रिय छ। तर एक्सप्रेस पूरानो भइसक्यो। यसमा धेरै अनावश्यक वजन (Bloat) छ र यो मोडर्न एउटा (Modern) async/await सँग सधैँ राम्रोसँग काम गर्दैन।

Dolphin को जन्म तीनवटा मुख्य कारणले भएको हो:

Native Speed: कुनै पनि बाहिरी लाइब्रेरी बिना नेटिभ http मा चल्ने।

Context-First: req र res लाई एउटै 'Context' (ctx) मा मिलाएर कोडलाई सफा राख्ने।

Total Modularity: तपाईंले Auth चाहनुहुन्छ? Auth मोड्युल मात्र लोड गर्नुहोस्।

पर्फर्मेन्स बेन्चमार्क Framework Requests Per Second (RPS) Latency (avg) Express.js ~15,000 10ms Fastify ~35,000 2ms Dolphin (v2.0) ~45,000+ 1.5ms १. सेटअप र वातावरण (Setup & Environment) १.१ आवश्यक चिजहरू (Prerequisites) Node.js: v18.x वा सोभन्दा माथि

TypeScript: Dolphin टाइप-सेफ छ, त्यसैले TS सिफारिस गरिन्छ।

१.२ प्रोजेक्ट सुरु गर्ने bash mkdir dolphin-master-app && cd dolphin-master-app npm init -y npm install dolphin-server-modules mongoose zod ioredis npm install -D typescript ts-node @types/node nodemon १.३ TypeScript कन्फिगर (tsconfig.json) json { "compilerOptions": { "target": "ES2022", "module": "CommonJS", "outDir": "./dist", "rootDir": "./src", "strict": true, "esModuleInterop": true, "skipLibCheck": true, "forceConsistentCasingInFileNames": true } } २. कोर सर्भर इन्जिन (Core Server Engine) २.१ सर्भर कसरी बनाउने? typescript import { createDolphinServer } from 'dolphin-server-modules/server';

const app = createDolphinServer();

app.listen(3000, () => { console.log("Dolphin Engine Active! 🐬 "); }); ३. Unified Context (ctx) Deep Dive ३.१ ctx भित्र के के हुन्छ? typescript app.get('/test', (ctx) => { // ctx.params, ctx.query, ctx.body, ctx.req, ctx.res }); मुख्य प्रोपर्टीहरू:

ctx.req: नेटिभ http.IncomingMessage

ctx.params: URL प्यारामिटरहरू

ctx.query: Query String

ctx.body: POST/PUT डाटा

मुख्य मेथडहरू:

ctx.json(obj): JSON डाटा पठाउन

ctx.status(code): HTTP स्टेटस कोड

ctx.header(key, value): रेस्पोन्स हेडर

३.२ अटो-JSON रेस्पोन्स [v1.4.7+] typescript // पुरानो: app.get('/old', (ctx) => { ctx.json({ ok: true }); });

// नयाँ: app.get('/new', (ctx) => { return { ok: true }; }); ४. एडभान्स्ड राउटिङ (Advanced Routing) typescript // Static Route app.get('/ping', (ctx) => ({ msg: 'pong' }));

// Dynamic Route app.get('/users/:id', (ctx) => ({ userId: ctx.params.id }));

// Route Prefixing app.group('/api/v1', (group) => { group.get('/users', (ctx) => ([])); }); ५. मिडलवेयर आर्किटेक्चर (Middleware Architecture) typescript // Global Middleware app.use((ctx, next) => { console.log(`${ctx.req.method} ${ctx.req.url}`); if (next) next(); });

// Multi-handler Support const isAdmin = (ctx, next) => { if (ctx.req.user?.role === 'admin') next(); else ctx.status(403).json({ error: "Denied" }); };

app.get('/admin', isAdmin, (ctx) => ({ message: "Welcome Admin" }));

// Express Middleware Compatible import cors from 'cors'; app.use(cors()); ६. डेटाबेस एड्याप्टर (Database Adapters) typescript import mongoose from 'mongoose'; import { createMongooseAdapter } from 'dolphin-server-modules/adapters/mongoose';

const User = mongoose.model('User', new mongoose.Schema({ username: String, email: String }));

const db = createMongooseAdapter({ User });

app.get('/users', async (ctx) => { return await db.User.find(); }); ७. Auth मोड्युल मास्टरक्लास (Auth Module) typescript import { createAuth } from 'dolphin-server-modules/auth';

const auth = createAuth({ secret: 'SUPER_SECRET_KEY', tokenExpiry: '1h', refreshExpiry: '7d' });

// Protect routes app.get('/dashboard', auth.middleware(), (ctx) => { return { user: ctx.req.user }; });

// 2FA Support app.post('/auth/2fa/setup', auth.middleware(), async (ctx) => { return await auth.setup2FA(ctx.req.user.id); }); ८. अटोमेटेड CRUD इन्जिन (Automated CRUD) typescript import { createCrudController } from 'dolphin-server-modules/curd';

const userCrud = createCrudController(db.User);

app.get('/api/users', userCrud.getAll); app.post('/api/users', userCrud.create); app.get('/api/users/:id', userCrud.getOne); app.put('/api/users/:id', userCrud.update); app.delete('/api/users/:id', userCrud.delete); ९. Zod भ्यालिडेसन (Zod Validation) typescript import { z } from 'zod'; import { validate } from 'dolphin-server-modules/middleware/zod';

const UserSchema = z.object({ name: z.string().min(3), email: z.string().email() });

app.post('/users', validate(UserSchema), (ctx) => { return { success: true, data: ctx.body }; }); १०. रियल-वर्ल्ड प्रोजेक्ट: DolphinStore API typescript // src/index.ts import { createDolphinServer } from 'dolphin-server-modules/server'; import { createAuth } from 'dolphin-server-modules/auth'; import { createMongooseAdapter } from 'dolphin-server-modules/adapters/mongoose'; import { createCrudController } from 'dolphin-server-modules/curd'; import mongoose from 'mongoose';

const app = createDolphinServer(); const auth = createAuth({ secret: 'DOLPHIN_STORE_SECRET' });

const Product = mongoose.model('Product', new mongoose.Schema({ name: String, price: Number, stock: { type: Number, default: 0 } }));

const db = createMongooseAdapter({ Product }); const productCrud = createCrudController(db.Product);

app.get('/products', productCrud.getAll); app.post('/products', auth.middleware(), productCrud.create); app.get('/products/:id/stock', async (ctx) => { const product = await Product.findById(ctx.params.id); if (!product) return ctx.status(404).json({ error: "Not found" }); return { stock: product.stock }; });

app.listen(8080, () => console.log("DolphinStore live on 8080! 🛒")); ११. स्केलिङ र पर्फर्मेन्स (Scaling) typescript import cluster from 'cluster'; import os from 'os';

if (cluster.isPrimary) { os.cpus().forEach(() => cluster.fork()); } else { const app = createDolphinServer(); app.listen(3000); } १२. टेस्टिङ र डेभप्स (Testing) typescript import request from 'supertest'; import { createDolphinServer } from 'dolphin-server-modules/server';

const app = createDolphinServer(); app.get('/test', (ctx) => ({ ok: true }));

test('GET /test', async () => { const res = await request(app.server).get('/test'); expect(res.body.ok).toBe(true); }); १३. RealtimeCore v2.0: रियलटाइम पब/सब मास्टरक्लास (पूर्ण अपडेटेड) ⚠️ महत्त्वपूर्ण: यो सेक्सन Dolphin Framework v2.0 को पूर्ण RealtimeCore अनुसार अपडेट गरिएको छ। तपाईंको Core मा pubPush, subPull, pubFile, subFile, Resume, P2P, Device Management जस्ता एडभान्स्ड फिचरहरू समावेश छन्।

RealtimeCore Dolphin को उच्च-प्रदर्शन युनिफाइड पब/सब बस हो। यो IoT डिभाइसहरू, वेबसकेटहरू, र माइक्रोसर्भिसहरू बीच रियलटाइम डाटा संचारको लागि डिजाइन गरिएको छ।

१३.१ v2.0 का नयाँ फिचरहरू फिचर विवरण pubPush / subPull अति उच्च गतिको डाटाको लागि (IoT Sensors, Live Graphs) - No JSON.stringify, No Redis pubFile / subFile ठूला फाइलहरू टुक्रा-टुक्रा (64KB chunks) मा पठाउने - Resume Support सहित resumeFile पहिले रोकिएको ठाउँबाट फाइल डाउनलोड पुनः सुरु गर्ने Device Management isOnline(), isReady(), sendTo(), kick(), broadcastToGroup() Private Messaging privateSub(), privatePub() - सुरक्षित संचार P2P Streaming पीयर-टु-पीयर डाटा सेयरिङ Auto Cleanup ६० सेकेन्ड इन्याक्टिभ डिभाइस आफै हट्छ High-Freq Buffers प्रति टपिक १०० वटा मात्र बफर (Memory efficient) १३.२ स्थापना (Installation) bash npm install dolphin-server-modules ioredis १३.३ Device Management (डिभाइस व्यवस्थापन) RealtimeCore v2.0 ले पूर्ण डिभाइस व्यवस्थापन क्षमता प्रदान गर्दछ:

typescript import { RealtimeCore } from 'dolphin-server-modules/realtime';

const rt = new RealtimeCore({ debug: true, enableJSONCache: true });

// WebSocket सर्भरसँग integrate गर्ने import WebSocket from 'ws'; const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws, req) => { const deviceId = req.headers['device-id'] as string || `device-${Date.now()}` ;

// डिभाइस रजिस्टर गर्ने (metadata सहित) rt.register(deviceId, ws, { type: 'sensor', location: 'kathmandu', group: 'temperature-sensors' });

ws.on('message', async (data: Buffer) => { await rt.handle(data, ws, deviceId); });

ws.on('close', () => { rt.unregister(deviceId); }); });

// डिभाइस अनलाइन छ कि छैन चेक गर्ने console.log(rt.isOnline('device-123')); // true/false console.log(rt.isReady('device-123')); // socket ready छ कि छैन

// डिभाइसलाई सिधै मेसेज पठाउने (बिना Pub/Sub) rt.sendTo('device-123', { type: 'PING', message: 'Are you alive?' });

// खराब डिभाइसलाई किक गर्ने rt.kick('bad-device', 'Unauthorized access detected');

// सबै अनलाइन डिभाइसको लिस्ट const devices = rt.getOnlineDevices(); // Output: [{ id: 'device-123', lastSeen: 123456, group: 'temperature-sensors' }]

// कुनै विशेष ग्रुपलाई मात्र ब्रोडकास्ट rt.broadcastToGroup('temperature-sensors', { command: 'read_temperature' });

// डिभाइसलाई पिंग गर्ने rt.ping('device-123'); १३.४ High-Frequency Messaging (pubPush/subPull) IoT Sensors, Live Graphs, Real-time Stock Data को लागि अति छिटो messaging:

typescript // सेन्सरबाट हाई-फ्रिक्वेन्सी डाटा पठाउने (pubPush) // यो method ले JSON.stringify, ACL, Redis सबै बाइपास गर्छ rt.pubPush('sensors/temperature/room1', Buffer.from([0x1A, 0x2B, 0x3C, 0x4D])); rt.pubPush('sensors/humidity/room1', { value: 65.5, unit: '%' }); // Auto JSON

// क्लाइन्टले डाटा मागेपछि मात्र पाउने (subPull) - Data Saving // क्लाइन्टले माग्दा पछिल्लो 10 वटा डाटा पाउँछ rt.subPull('device-123', 'sensors/temperature/room1', 10);

// pubPush डाटा अटोम्याटिक बफरमा सेभ हुन्छ // subPull ले बफरबाट पछिल्लो डाटा निकाल्छ १३.५ File Transfer with Resume (pubFile/subFile) ठूला फाइलहरू टुक्रा-टुक्रा गरेर पठाउने। रिज्युम सपोर्ट सहित:

typescript // सर्भरले फाइल पब्लिस गर्ने तयारी (pubFile) const fileId = 'patient-report-123'; const metadata = rt.pubFile(fileId, '/path/to/patient-report.pdf', 64 * 1024); // 64KB chunks

if (metadata) { console.log(`File registered: ${metadata.name}, Total chunks: ${metadata.totalChunks}`); }

// क्लाइन्टले फाइल डाउनलोड गर्ने (subFile) // startChunk = 0 भनेको सुरुदेखि नै डाउनलोड गर्ने await rt.subFile('device-123', fileId, 0);

// Resume: पहिले रोकिएको ठाउँबाट पुनः सुरु गर्ने const lastChunk = rt.getFileProgress('device-123', fileId); if (lastChunk >= 0) { console.log(`Resuming from chunk ${lastChunk + 1}`); await rt.resumeFile('device-123', fileId); }

// फाइलको जानकारी लिने const fileInfo = rt.getFileInfo(fileId); console.log(`File: ${fileInfo?.name}, Size: ${fileInfo?.size} bytes`);

// सबै उपलब्ध फाइलहरूको सूची const files = rt.listFiles(); १३.६ P2P Streaming सर्भर मार्फत पीयरहरू बीच सिधै डाटा सेयरिङ:

typescript // फाइलको उपलब्धता सबै पीयरलाई जानकारी दिने rt.announceToPeers('file-123', 'device-A');

// फाइल कुन-कुन पीयरसँग छ हेर्ने const peers = rt.getPeersForFile('file-123'); console.log(`File available at: ${peers}`);

// पीयरबाट सिधै डाटा माग गर्ने rt.requestFromPeer('device-B', 'device-A', 'file-123', 5); // chunk 5

// पीयरलाई सिधै डाटा पठाउने (Server Pass-through) rt.sendToPeer('device-A', 'device-B', { chunk: 5, data: buffer }); १३.७ Private Messaging सुरक्षित निजी संचारको लागि:

typescript // क्लाइन्टले आफ्नो प्राइभेट च्यानल सुन्ने rt.privateSub('device-123', (msg) => { console.log('Private message received:', msg); });

// कसैलाई प्राइभेट मेसेज पठाउने rt.privatePub('device-456', { secret: 'data', action: 'alert' });

// नोट: privatePub ले `phone/signaling/{targetId}` टपिकमा publish गर्छ १३.८ Basic Pub/Sub (Retro-compatible) पुरानो MQTT-शैली pub/sub पनि काम गर्छ:

typescript // टपिक सब्सक्राइब गर्ने (वाइल्डकार्ड सपोर्ट) rt.subscribe('sensors/temperature/+', (payload, topic) => { console.log(`Temperature data: ${payload.value}°C from ${topic}`); });

// टपिकमा पब्लिस गर्ने rt.publish('sensors/temperature/room1', { value: 23.5, unit: 'celsius' });

// रिटेन्ड मेसेज (नयाँ सब्सक्राइबरले पाउने) rt.publish('config/system', { mode: 'active' }, { retain: true, ttl: 3600000 });

// वाइल्डकार्ड म्याचिङ // + (सिङ्गल लेभल) - एउटा सेग्मेन्ट मात्र rt.subscribe('sensors/+/temperature', (data) => { // म्याच: sensors/room1/temperature, sensors/room2/temperature });

// # (मल्टी लेभल) - सबै सेग्मेन्टहरू rt.subscribe('sensors/#', (data) => { // म्याच: sensors/temp, sensors/room1/humidity, sensors/building/floor/room/temp }); १३.९ Redis Scaling एकाधिक सर्भरहरू बीच मेसेज सिंक गर्न:

typescript // सर्भर १ const rt1 = new RealtimeCore({ redisUrl: 'redis://localhost:6379' });

// सर्भर २ (अर्को मेसिनमा) const rt2 = new RealtimeCore({ redisUrl: 'redis://localhost:6379' });

// rt1 मा पब्लिस गरेको मेसेज rt2 मा पुग्छ rt1.publish('global/event', { message: 'Hello from Server 1' }); १३.१० ACL (Access Control List) typescript const rt = new RealtimeCore({ acl: { canSubscribe: (deviceId, topic) => { // डिभाइसले केवल आफ्नै टपिकमा सब्सक्राइब गर्न पाउँछ return topic.startsWith(`devices/${deviceId}`) || topic === 'public/#'; }, canPublish: (deviceId, topic) => { // सेन्सर डिभाइसले sensors/ मात्र पब्लिस गर्न पाउँछ if (deviceId.startsWith('sensor-')) return topic.startsWith('sensors/'); return true; } } }); १३.११ प्लगिन सिस्टम (Plugin System) typescript import { RealtimePlugin, RealtimeContext } from 'dolphin-server-modules/plugins';

// MQTT प्लगिन बनाउने const mqttPlugin: RealtimePlugin = { name: 'mqtt-bridge', match: (ctx) => ctx.raw[0] === 0x10, // MQTT CONNECT प्याकेट decode: (raw) => { // MQTT डिकोडिङ लजिक return { type: 'mqtt', payload: raw }; }, onMessage: (ctx) => { console.log('MQTT मेसेज आयो:', ctx.payload); } };

rt.use(mqttPlugin); १३.१२ पूर्ण उदाहरण: हस्पिटल मोनिटरिङ सिस्टम यो उदाहरणले हस्पिटलमा Telephone System र Equipment Monitoring को लागि पूर्ण real-time solution देखाउँछ:

typescript // hospital-realtime.ts import { RealtimeCore } from 'dolphin-server-modules/realtime'; import WebSocket from 'ws'; import mongoose from 'mongoose';

// MongoDB Schema const TelephoneSchema = new mongoose.Schema({ deviceId: String, ip: String, extension: String, status: { type: String, enum: ['idle', 'busy', 'offline', 'ringing'] }, busy: { isBusy: Boolean, occupiedBy: String, since: Date }, lastSeen: Date });

const EquipmentSchema = new mongoose.Schema({ equipmentId: String, name: String, type: String, status: { type: String, enum: ['online', 'offline', 'in_use', 'maintenance'] }, busy: { isBusy: Boolean, occupiedBy: String, since: Date }, patientInfo: { patientId: String, roomNo: String, bedNo: String }, lastSeen: Date });

const Telephone = mongoose.model('Telephone', TelephoneSchema); const Equipment = mongoose.model('Equipment', EquipmentSchema);

// RealtimeCore Setup const rt = new RealtimeCore({ maxMessageSize: 1024 * 1024, enableJSONCache: true, debug: true });

// 1. Device Registration Handler async function handleDeviceConnection(ws: WebSocket, deviceId: string, deviceType: string) { rt.register(deviceId, ws, { type: deviceType, registeredAt: Date.now() });

// Send current status immediately const status = await getCurrentStatus(deviceId, deviceType); rt.sendTo(deviceId, { type: 'INIT_STATUS', data: status });

ws.on('message', async (data) => { await rt.handle(data as Buffer, ws, deviceId); await updateHeartbeat(deviceId, deviceType); });

ws.on('close', () => { rt.unregister(deviceId); markDeviceOffline(deviceId, deviceType); }); }

// 2. Telephone Status Update (Real-time) rt.subscribe('telephone/+/status', async (data, topic) => { const deviceId = topic.split('/')[1]; await Telephone.findOneAndUpdate({ deviceId }, { status: data.status, lastSeen: new Date() });

// Broadcast to all monitoring clients rt.broadcast('monitor/telephone/update', { deviceId, status: data.status });

// Special alert for busy if (data.status === 'busy') { rt.publish('alert/telephone/busy', { deviceId, callWith: data.callWith }); } });

// 3. Equipment Reservation with Resume Support rt.subscribe('equipment/reserve', async (data) => { const { equipmentId, patientId, roomNo } = data;

const equipment = await Equipment.findOne({ equipmentId }); if (equipment?.busy?.isBusy) { rt.sendTo(data.deviceId, { type: 'RESERVE_FAILED', reason: `Equipment busy with ${equipment.busy.occupiedBy}` }); return; }

await Equipment.findOneAndUpdate({ equipmentId }, { 'busy.isBusy': true, 'busy.occupiedBy': patientId, 'busy.since': new Date(), patientInfo: { patientId, roomNo }, status: 'in_use' });

rt.publish(`equipment/${equipmentId}/reserved` , { patientId, roomNo }); rt.sendTo(data.deviceId, { type: 'RESERVE_SUCCESS', equipmentId }); });

// 4. File Transfer for Medical Reports rt.subscribe('report/request', async (data) => { const { patientId, reportType, deviceId } = data; const reportPath = `/reports/${patientId}/${reportType}.pdf` ; const fileId = `report-${patientId}-${Date.now()}` ;

const metadata = rt.pubFile(fileId, reportPath); if (metadata) { rt.sendTo(deviceId, { type: 'REPORT_READY', fileId, name: metadata.name, size: metadata.size, totalChunks: metadata.totalChunks }); } });

// 5. Resume interrupted file download rt.subscribe('report/resume', async (data) => { const { fileId, deviceId } = data; const lastChunk = rt.getFileProgress(deviceId, fileId);

if (lastChunk >= 0) { await rt.resumeFile(deviceId, fileId); rt.sendTo(deviceId, { type: 'RESUME_STARTED', fromChunk: lastChunk + 1 }); } });

// 6. Heartbeat Monitor (Auto offline detection) setInterval(async () => { const timeout = new Date(Date.now() - 10000); // 10 seconds

const offlineTelephones = await Telephone.updateMany({ lastSeen: { $lt: timeout }, status: { $ne: 'offline' } }, { status: 'offline' });

const offlineEquipment = await Equipment.updateMany({ lastSeen: { $lt: timeout }, status: { $ne: 'offline' } }, { status: 'offline' });

if (offlineTelephones.modifiedCount > 0 || offlineEquipment.modifiedCount > 0) { rt.publish('alert/offline', { telephones: offlineTelephones.modifiedCount, equipment: offlineEquipment.modifiedCount, timestamp: Date.now() }); } }, 5000);

// 7. Dashboard Stats (Real-time) rt.subscribe('dashboard/stats', async (data, deviceId) => { const [telephones, equipment] = await Promise.all([Telephone.find(), Equipment.find()]);

rt.sendTo(deviceId, { type: 'DASHBOARD_STATS', data: { telephones: { total: telephones.length, busy: telephones.filter(t => t.status === 'busy').length, offline: telephones.filter(t => t.status === 'offline').length }, equipment: { total: equipment.length, inUse: equipment.filter(e => e.busy?.isBusy).length, offline: equipment.filter(e => e.status === 'offline').length }, timestamp: Date.now() } }); });

// Helper functions async function updateHeartbeat(deviceId: string, type: string) { const Model = type === 'telephone' ? Telephone : Equipment; const idField = type === 'telephone' ? 'deviceId' : 'equipmentId'; await Model.findOneAndUpdate({ [idField]: deviceId }, { lastSeen: new Date(), status: 'online' }); }

async function markDeviceOffline(deviceId: string, type: string) { const Model = type === 'telephone' ? Telephone : Equipment; const idField = type === 'telephone' ? 'deviceId' : 'equipmentId'; await Model.findOneAndUpdate({ [idField]: deviceId }, { status: 'offline' }); }

async function getCurrentStatus(deviceId: string, type: string) { const Model = type === 'telephone' ? Telephone : Equipment; const idField = type === 'telephone' ? 'deviceId' : 'equipmentId'; return await Model.findOne({ [idField]: deviceId }); }

// Start WebSocket Server const wss = new WebSocket.Server({ port: 8080 }); wss.on('connection', (ws, req) => { const url = new URL(req.url!, `http://${req.headers.host}`); const deviceId = url.searchParams.get('id')!; const deviceType = url.searchParams.get('type')!; // 'telephone' or 'equipment'

handleDeviceConnection(ws, deviceId, deviceType); });

// Start HTTP Server with Dolphin import { createDolphinServer } from 'dolphin-server-modules/server'; const app = createDolphinServer();

app.get('/stats', (ctx) => { return rt.getStats(); });

app.listen(3000, () => { console.log('🏥 Hospital Realtime System Running'); console.log('📊 Stats:', rt.getStats()); });

console.log('Hospital Monitoring System Active! 🏥 '); १३.१३ API रेफरेन्स (RealtimeCore v2.0) Constructor Options:

typescript interface RealtimeCoreConfig { maxMessageSize?: number; // डिफल्ट: 256KB redisUrl?: string; // Redis URL (ऐच्छिक) acl?: { // Access Control canSubscribe: (deviceId, topic) => boolean; canPublish: (deviceId, topic) => boolean; }; enableJSONCache?: boolean; // JSON क्यास सक्षम useBinaryProtocol?: boolean; // बाइनरी प्रोटोकल debug?: boolean; // डिबग मोड maxBufferPerTopic?: number; // pubPush बफर साइज (डिफल्ट: 100) defaultChunkSize?: number; // File chunk size (डिफल्ट: 64KB) enableP2P?: boolean; // P2P सक्षम } Methods:

Method Description subscribe(topic, fn, deviceId?) टपिकमा सब्सक्राइब publish(topic, payload, opts?, deviceId?) टपिकमा पब्लिस pubPush(topic, payload) हाई-फ्रिक्वेन्सी पब्लिस (No JSON) subPull(deviceId, topic, count?) बफरबाट डाटा तान्ने pubFile(fileId, filePath, chunkSize?) फाइल ट्रान्सफर तयारी subFile(deviceId, fileId, startChunk?) फाइल डाउनलोड resumeFile(deviceId, fileId) रोकिएको डाउनलोड पुनः सुरु getFileProgress(deviceId, fileId) डाउनलोड प्रगति हेर्ने listFiles() सबै उपलब्ध फाइलहरू register(deviceId, socket?, metadata?) डिभाइस रजिस्टर unregister(deviceId) डिभाइस हटाउने isOnline(deviceId) डिभाइस अनलाइन छ कि छैन isReady(deviceId) सकेट तयार छ कि छैन sendTo(deviceId, payload) डिभाइसमा सिधै पठाउने kick(deviceId, reason?) डिभाइस हटाउने broadcastToGroup(groupName, payload) ग्रुपमा ब्रोडकास्ट getOnlineDevices() सबै अनलाइन डिभाइस ping(deviceId) डिभाइस पिंग गर्ने privateSub(deviceId, fn) प्राइभेट च्यानल सुन्ने privatePub(targetId, payload, opts?) प्राइभेट मेसेज पठाउने announceToPeers(fileId, deviceId) P2P घोषणा getPeersForFile(fileId) पीयर सूची handle(raw, socket?, deviceId?) कच्चा डाटा प्रोसेस broadcast(topic, payload, opts?) सबै डिभाइसमा पठाउने use(plugin) प्लगिन थप्ने getStats() स्ट्याटिस्टिक्स destroy() सबै रिसोर्स क्लिनअप १४. इन्डिपेन्डेन्ट राउटिङ (Independent Routing) typescript // userRoutes.ts import { createDolphinRouter } from 'dolphin-server-modules/router'; export const userRouter = createDolphinRouter();

userRouter.get('/profile', (ctx) => ({ name: "Ram", email: "ram@example.com" })); userRouter.post('/update', (ctx) => ({ success: true }));

// main.ts import { createDolphinServer } from 'dolphin-server-modules/server'; import { userRouter } from './userRoutes';

const app = createDolphinServer(); app.use('/users', userRouter); // /users/profile, /users/update १५. स्वतन्त्र अटो-स्वैगर जेनेरेसन (Auto-Swagger Generation) typescript import { z } from 'zod'; import { generateSwagger, serveSwaggerUI } from 'dolphin-server-modules/swagger'; import { createDolphinServer } from 'dolphin-server-modules/server';

const app = createDolphinServer();

const UserSchema = z.object({ name: z.string().min(3), email: z.string().email(), age: z.number().optional() });

const apiDocs = generateSwagger({ title: "DolphinStore API", version: "1.0.0", modules: [{ path: "/users", method: "post", schema: UserSchema, summary: "Create new user", tags: ["Users"] }] });

app.get('/docs', (ctx) => { const html = serveSwaggerUI(apiDocs, "DolphinStore API"); return ctx.html(html); });

app.get('/docs/json', (ctx) => ctx.json(apiDocs)); १६. API रेफरेन्स (API Reference) Server Method Description app.listen(port, cb) सर्भर सुरु गर्ने app.get/patch/post/put/delete(path, ...handlers) HTTP मेथड app.use(path?, middleware/router) मिडलवेयर वा राउटर app.group(prefix, callback) रूट ग्रुपिङ RealtimeCore v2.0 Method Description rt.subscribe(topic, fn, deviceId?) सब्सक्राइब rt.publish(topic, payload, opts?, deviceId?) पब्लिस rt.pubPush(topic, payload) हाई-फ्रिक्वेन्सी पब्लिस rt.subPull(deviceId, topic, count?) बफरबाट तान्ने rt.pubFile(fileId, filePath, chunkSize?) फाइल तयारी rt.subFile(deviceId, fileId, startChunk?) फाइल डाउनलोड rt.resumeFile(deviceId, fileId) डाउनलोड पुनः सुरु rt.register(deviceId, socket?, metadata?) डिभाइस रजिस्टर rt.sendTo(deviceId, payload) सिधै पठाउने rt.kick(deviceId, reason?) डिभाइस हटाउने rt.privateSub(deviceId, fn) प्राइभेट सब्सक्राइब rt.privatePub(targetId, payload, opts?) प्राइभेट पब्लिस rt.getStats() स्ट्याटिस्टिक्स rt.destroy() क्लिनअप १७. Universal Signaling (WebRTC & IoT) Dolphin v1.6.0 बाट शून्य डिपेन्डेन्सी सहितको Universal Signaling Module आएको छ, जसले WebRTC र IoT Control लाई एउटै API मा ह्यान्डल गर्छ।

typescript import { createSignaling } from 'dolphin-server-modules/signaling'; import { RealtimeCore } from 'dolphin-server-modules/realtime';

const rt = new RealtimeCore(); const signaling = createSignaling(rt);

// 1. WebRTC Call (For Phones/Apps) await signaling.invite('user1', 'user2', { sdp: 'offer_data' });

// 2. IoT / Medical Command await signaling.sendCommand('DoctorApp', 'Machine_01', { action: 'START' });

१८. Folder Structure र Architecture बेस्ट प्राक्टिस (Zero to Scale) एउटा ठूलो सङ्गठन वा IoT एप्लिकेसन डिजाइन गर्दा प्रोजेक्टलाई एउटै फाइलमा नराखी व्यवस्थित तरिकाले यसरी मिलाउनुपर्छ:

```
my-dolphin-app/
├── src/
│   ├── config/              # Database, Redis जस्ता कन्फिगरेसन
│   ├── models/              # Mongoose/DB स्किमाहरू (Product.ts, SensorData.ts)
│   ├── controllers/         # लजिक र CRUD ह्यान्डलर (UserCtrl.ts, IoT_Ctrl.ts)
│   ├── middlewares/         # ZodValidation, AuthGuard, ErrorHandler
│   ├── routes/              # apiRouter (Independent Routing)
│   ├── realtime/            # IoT र WebSocket का इभेन्ट-ह्यान्डलर (pub/sub)
│   ├── app.ts               # Dolphin Server र WebSockets (wss) को सेटअप
│   └── index.ts             # सर्भर सुरु गर्ने (app.listen)
├── package.json
└── tsconfig.json
```

यो तरिका अपनाउँदा तपाइँको REST API र IoT को RealtimeCore कहिल्यै पनि नराम्रोसँग जेलिँदैन (Tight Coupling हुँदैन)।

१९. Global Error Handling (सर्भर क्रयास हुनबाट बचाउने) Dolphin मा तपाईंले लेखेको कुनै कस्टम लजिकले Error फाले पनि सर्भर क्रयास हुनुहुँदैन। यसको लागि ग्लोबल मिडलवेयर (Global Middleware) प्रयोग गर्नुपर्छ:

```
app.use(async (ctx, next) => {
  try {
    if (next) await next();
  } catch (error) {
    console.error("🔥 SYSTEM ERROR:", error.message);
    // क्लाईन्टलाई नराम्रो HTML को सट्टा राम्रो JSON पठाउने
    ctx.status(500).json({
      success: false,
      message: "Internal Server Error",
      error: process.env.NODE_ENV !== 'production' ? error.stack : undefined
    });
  }
});
```

२०. Frontend सँग जोड्ने (React & IoT WebSockets) फ्रन्टइन्ड (Frontend) बाट Dolphin API लाई कल गर्न तपाईंले सुरुमा CORS अन गर्नुपर्छ।

CORS Setup:

```
import cors from 'cors';
app.use(cors({ origin: 'http://localhost:3000', credentials: true }));
```

React बाट IoT WebSocket जोड्ने:

```
import { useEffect, useState } from 'react';

function Dashboard() {
  const [temp, setTemp] = useState(0);

  useEffect(() => {
    // 1. Dolphin को WebSocket मा कनेक्ट गर्ने
    const ws = new WebSocket('ws://localhost:8080?id=React_Dashboard&type=monitor');
```

```
    ws.onmessage = (event) => {
      const parsed = JSON.parse(event.data);
      if(parsed.topic === 'sensors/temperature') {
         setTemp(parsed.payload.value);
      }
    };

    // 2. Dolphin लाई कुन टपिक सुन्ने भनेर Subscribe मेसेज पठाउने
    ws.onopen = () => {
       ws.send(JSON.stringify({ action: 'subscribe', topic: 'sensors/temperature' }));
    };

    return () => ws.close();
  }, []);

  return <div><h1>लाइभ तापक्रम: {temp} °C 🌡️</h1></div>;
}
```

२१. Production Deployment (सर्भर लाइभ गर्ने) तपाईंको Dolphin सर्भर रकेट जस्तै तयार छ। अब यसलाई लाइभ (World Wide Web) मा राख्न PM2 प्रयोग गरिन्छ जसले क्रयास भएको खण्डमा तुरुन्त (0.1ms) सर्भरलाई अटो-रिस्टार्ट (Auto-Restart) गर्छ।

PM2 इन्स्टल र सुरु गर्न:

```
# 1. PM2 लाई Global रूपमा राख्ने
npm install -g pm2

# 2. तपाईंको Code लाई Build गर्ने
npm run build

# 3. PM2 द्वारा Background मा सर्भर सुरु गर्ने
pm2 start dist/index.js --name "dolphin-iot-backend"

# 4. सर्भरका Logs (Error/Info) हेर्ने
pm2 logs dolphin-iot-backend
```

२२. निष्कर्ष (Conclusion) बधाई छ! तपाईंले Dolphin Framework v2.0 को Master Guide पूरा गर्नुभयो। अब तपाईं:

✅ हाई-पर्फर्मेन्स API सर्भर बनाउन ✅ अटोमेटेड CRUD र भ्यालिडेसन प्रयोग गर्न ✅ RealtimeCore v2.0 सँग पूर्ण रियलटाइम एप्लिकेसन बनाउन ✅ pubPush/subPull सँग हाई-फ्रिक्वेन्सी डाटा ह्यान्डल गर्न ✅ pubFile/subFile सँग ठूला फाइलहरू रिज्युम सपोर्ट सहित ट्रान्सफर गर्न ✅ Device Management सँग डिभाइसहरू ट्र्याक गर्न ✅ Redis सँग मल्टिपल सर्भर स्केल गर्न ✅ अटो-जेनेरेटेड Swagger डकुमेन्टेसन बनाउन

सक्नुहुन्छ।

Happy Coding! 🇳🇵 नेपालबाट विश्वस्तरको सफ्टवेयर बनाऔं!